Hospital Queue Management System

A smarter way to register patients, issue tokens, and track doctor queues in real time.

React • Node.js / Express • MongoDB

TEAM -2

U. VIDYA SRI

S.AISHWARYA

T.RANGANATH SWAMY

M.HEMANTH REDDY

Long waiting times

Patients often do not know their place in the queue or how long they may wait.

Manual coordination

Paper tokens and verbal updates can lead to confusion, duplicate entries and missed turns.

Limited visibility

Reception staff and doctors need a shared view of queue status and patient flow.

Project goal

Build a MERN-based web application to manage patients, doctors, departments, tokens and queue statuses through a clear dashboard and REST APIs.

Proposed Solution

1. Register

Create or find a patient record, select department and doctor, then issue a unique token.

2. Manage queue

Reception can call the next token, mark a patient waiting, in consultation, completed or cancelled.

3. Monitor

Dashboard shows current token, waiting count, doctor availability and queue status.

Users & Permissions

Administrator

- Manage staff accounts
- Add/edit departments and doctors
- View all queues and reports
- Configure system settings

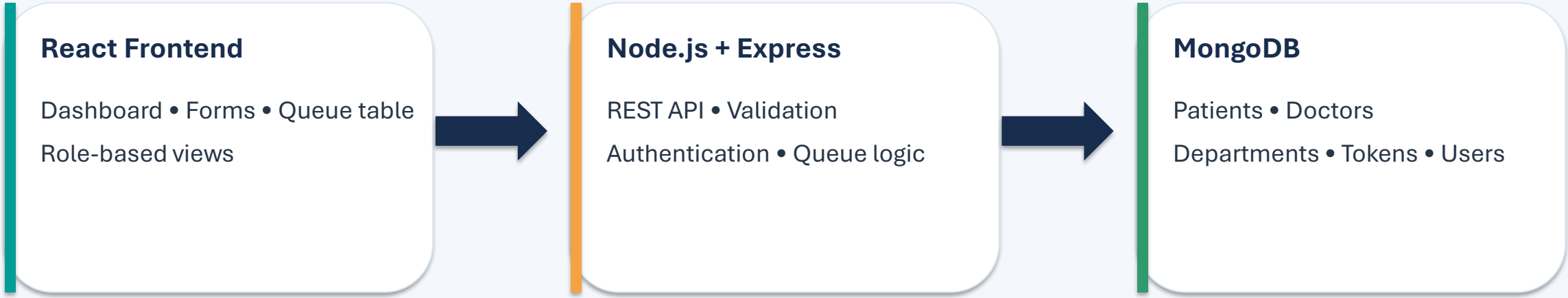
Receptionist

- Register/search patients
- Create queue token
- View waiting list
- Call next / update token status

Doctor

- View assigned queue
- Call next patient
- Mark consultation in progress
- Complete or defer a token

System Architecture



Request flow: React → HTTP/JSON → Express routes/controllers → Mongoose models → MongoDB

Security basics: input validation, password hashing, JWT/session protection, role checks, CORS configuration and environment variables.

MongoDB Data Model

Patient

name, phone, age/date Of Birth, gender,

Doctor

name, department Id, specialization, availability, active

Department

name, description, active

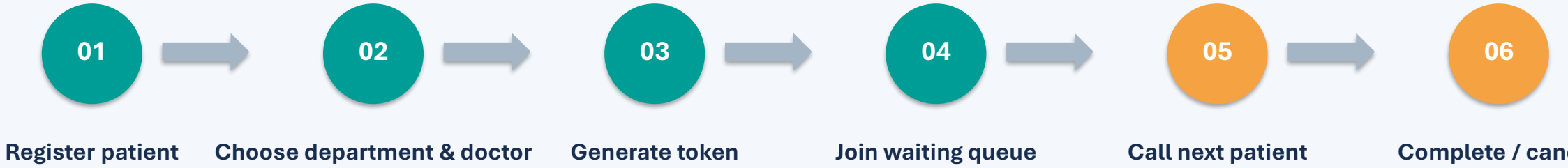
Token / QueueEntry

Token Number, patient Id, doctor Id, department Id, date, status, called At, completed At

User

name, email, password Hash, role, active

Queue Workflow



Queue statuses

Waiting → Called → In Consultation → Completed

Other outcomes

Cancelled / No-show; keep an audit timestamp where appropriate.

Security, Validation & Testing

Authentication

Hash passwords; issue secure JWTs or use sessions; protect private routes.

Authorization

Check roles on the server for every restricted action, not only in the UI.

Data validation

Validate IDs, required fields, phone/email formats and allowed status transitions.

Test checklist

- CRUD endpoints return expected status codes and clear error messages.
- Duplicate or invalid token generation is prevented for the same queue/day.
- Users cannot perform actions outside their role.
- Queue ordering and status updates persist after page refresh.

THANK YOU